MWF - Quiz 2026

Dokumentáció – 2025/2026 tavaszi félév

1. Fejlesztő adatai

Név: **Horváth Olivér**

Neptun kód: **D6018J**

2. Feladatleírás

2.1. Feladat összefoglalása

A feladat egy valós idejű, Kahoot jellegű kvízrendszer elkészítése kliens–szerver architektúrában. A rendszerben a felhasználók saját kvízeket hozhatnak létre, a résztvevők pedig egy aktív kvízhez csatlakozva meghatározott időkereten belül válaszolhatnak a szerver által vezérelt kérdésekre.

A webszolgáltatást REST architektúrát követő ASP.NET WebAPI alkalmazásként kell megvalósítani C# nyelven, MVC szemléletben. A rendszerhez egy résztvevői és egy adminisztrációs kliensfelület tartozik, amelyeknek együttműködve kell biztosítaniuk a teljes használatot és bemutathatóságot.

- A résztvevői felületen a felhasználók regisztrálhatnak, bejelentkezhetnek, illetve bejelentkezés nélkül is csatlakozhatnak egy aktív kvízhez egyedi azonosító megadásával.
- Az aktív kvíz során a kliens mindig az aktuális kérdést és annak válaszlehetőségeit jeleníti meg, egy kérdésre pontosan egy válasz adható le, kizárólag az aktív időszak alatt.
- A kérdés lezárása után a résztvevők megtekinthetik az adott kérdés eredményét.
- Az adminisztrációs felületen a bejelentkezett felhasználók megtekinthetik saját kvízeiket, új kvízt hozhatnak létre, valamint vezérelhetik a kvíz lebonyolítását.
- Egy kvíz létrehozásakor megadható a cím, a kérdések dinamikus száma, kérdésenként legalább két válaszlehetőség és a helyes válasz.
- A kérdések sorrendjét, aktiválását, lezárását és a következő kérdésre léptetést a szerver által vezérelt adminisztrációs folyamat szabályozza.
- A rendszernek biztosítania kell, hogy egy kvízben egyszerre tetszőleges számú résztvevő vehessen részt, és ugyanaz a felhasználó több kliensről is bejelentkezhessen.
- Az adatok relációs adatbázisban tárolódnak, kezelésük Entity Framework használatával történik, az authentikáció és authorizáció pedig ASP.NET Identity alapokra épül.
- A megoldásnak törekednie kell a biztonságos adatkezelésre, a felhasználóbarát működésre, a hibatűrésre, valamint a funkcionalitás integrációs tesztekkel történő ellenőrzésére.

2.2. Választott feladatok

- SignalR alapfeladat: a kérdések megjelenítése, lezárása és a kérdések közötti léptetés valós időben történik oldalfrissítés nélkül.
- OAuth2 alapú authentikáció: a hitelesítés refresh token flow mintával valósul meg, a hozzáférési tokenek JWT formátumúak.
- Szerkeszthető kvízek: az adminisztrációs felületen a saját, még nem elindított kvízek módosíthatók.

- Élő chat: a résztvevői felületen a kvízhez tartozó üzenetek valós időben jelennek meg, és a korábbi üzenetek is visszanézhetők.

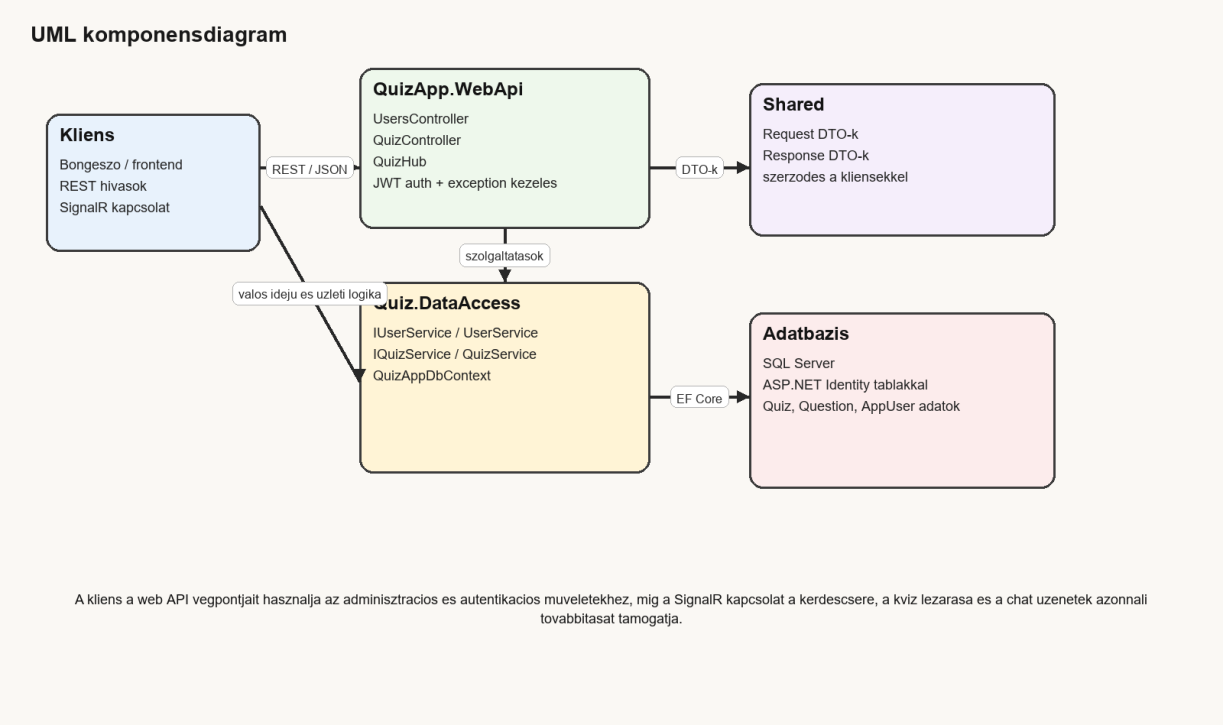
3. Funkcionális elemzés

- A rendszer két fő szerepkört kezel: kvízkészítő felhasználó és csatlakozó játékos.
- A felhasználó regisztrálhat, majd bejelentkezés után saját kvízeket hozhat létre és listázhat.
- A kvíz címéből és kérdéseiből áll; egy kérdés szöveget, válaszlehetőségeket és helyes válasz indexet tartalmaz.
- A még nem publikált kvíz szerkeszthető, így a kérdések és a cím módosíthatók.
- A publikálás során a rendszer egy egyedi, hatjegyű csatlakozási kódot generál.
- A játékos a csatlakozási kód és egy választott név megadásával léphet be a kvízbe.
- A kvíz tulajdonosa léptetheti a kérdéseket, lezárhatja az aktuális kérdést, majd befejezheti a teljes játékot.
- A valós idejű működést a SignalR biztosítja, amely azonnal értesíti a klienseket a játékállapot változásairól.
- A chat funkció lehetővé teszi, hogy a csatlakozott játékosok üzenetet küldjenek az adott kvízhez.
- A hitelesítés JWT bearer tokennel történik, a hozzáférési token megújítása refresh tokennel valósul meg.

4. Fejlesztői dokumentáció

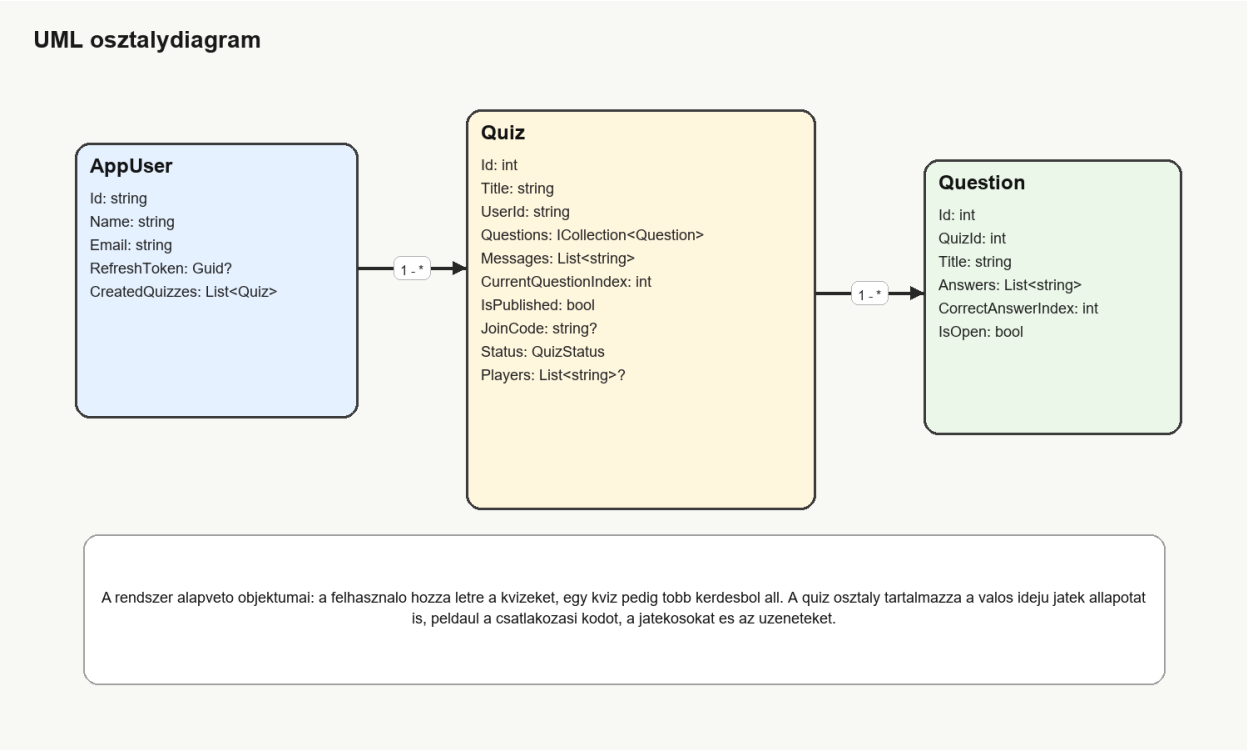
4.1. Statikus terv

A megoldás négy fő projektre bontott: a webes API rétegre, az adatelérési és üzleti logikai rétegre, a közös DTO-kat tartalmazó projekt részre, valamint a tesztprojektre.



1. ábra - A rendszer komponensei és kapcsolatai

A webes réteg a kérések fogadásáért, a hitelesítésért, az útvonalkezelésért és a SignalR hub közzétételéért felel. Az üzleti szabályok a szolgáltatásokban, az állandó adatok kezelése pedig az Entity Framework Core alapú adatelérési rétegben található.



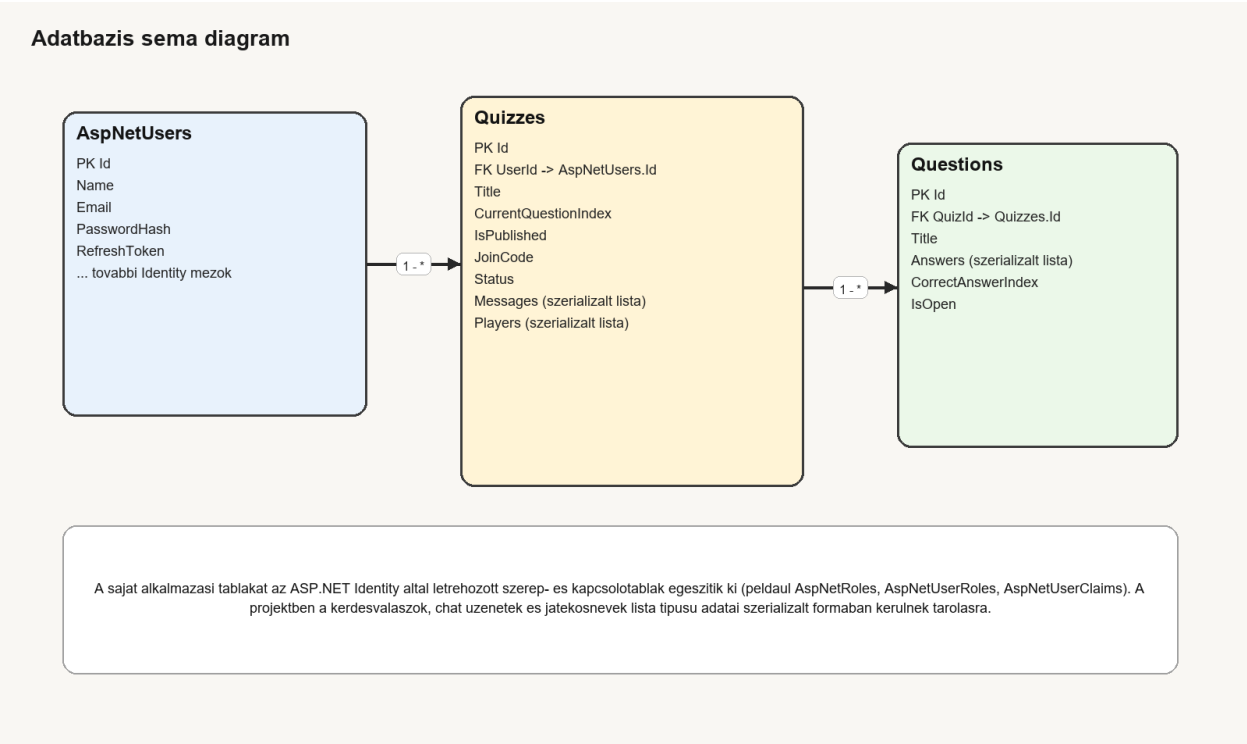
2. ábra - A legfontosabb alkalmazási osztályok

Az osztálydiagramon csak a lényegi domain elemek jelennek meg. A rendszer működését alapvetően az AppUser, Quiz és Question entitások írják le.

4.2. Adatbázis felépítése

A projekt SQL Server adatbázist használ Entity Framework Core segítségével. A felhasználókezelés az ASP.NET Identity sémájára épül, amelyet az alkalmazás saját Quiz és Question táblái egészítenek ki.

- AspNetUsers: a felhasználók adatai, valamint a refresh token tárolása.
- Quizzes: a kvíz törzsadatai, állapota, csatlakozási kódja, játékoslistája és üzenetei.
- Questions: a kérdések szövege, válaszai, helyes válasza és nyitott vagy lezárt állapota.
- Identity táblák: a szerepkörök és kapcsolatok tárolására szolgáló ASP.NET Identity segédtáblák.



3. ábra - A főbb adatbázistáblák kapcsolatai

A Quiz és a Question között egy-egy több kapcsolat található, ugyanígy egy felhasználó több kvíz létrehozója lehet.

4.3. A webszolgáltatás felülete

A rendszer REST alapú HTTP végpontokat és egy SignalR hubot tesz elérhetővé. A tulajdonosi műveletek hitelesítést igényelnek, a játékos oldali csatlakozási és chat műveletek viszont a join code alapján is használhatók.

Metódus	Végpont	Rövid leírás	Hitelesítés
POST	/users	Felhasználó regisztráció	nem
POST	/users/login	Bejelentkezés, JWT és refresh token kiadása	nem
POST	/users/logout	Kijelentkezés	igen
POST	/users/refresh	Új hozzáférési token kérése refresh tokennel	nem
GET	/users/{id}	Saját felhasználói adat lekérése	igen
POST	/quizzes	Új kvíz létrehozása	igen
POST	/quizzes/{id}/update	Meglévő, még nem publikált kvíz szerkesztése	igen
GET	/quizzes/{quizId}	Kvíz lekérése tulajdonosként	igen
GET	/quizzes/mine	Saját kvízek listázása	igen

GET	/quizzes/publish/{id}	Kvíz publikálása, csatlakozási kód létrehozása	igen
POST	/quizzes/join	Csatlakozás közzétett kvízhez	nem
POST	/quizzes/code	Kvíz lekérése csatlakozási kód alapján	nem
GET	/quizzes/{id}/next	Következő kérdés indítása	igen
GET	/quizzes/{id}/question/end	Aktuális kérdés lezárása	igen
GET	/quizzes/{id}/end	Kvíz befejezése	igen
POST	/quizzes/message	Chat üzenet küldése a kvízhez	nem

Elem	Név	Szerep
Hub	/hubs/quiz	A valós idejű kommunikáció belépési pontja
Kliens metódus	JoinQuizGroup(joinCode)	Csatlakozás egy kvízhez tartozó csoporthoz
Kliens metódus	LeaveQuizGroup(joinCode)	Kilépés a kvízcsoportból
Szerver esemény	QuestionChanged	Értesítés új kérdés indításakor
Szerver esemény	QuestionClosed	Értesítés kérdéslezáráskor, helyes válasz indexével
Szerver esemény	QuizEnded	Értesítés a kvíz befejezéséről
Szerver esemény	MessageSent	Új chat üzenet továbbítása

A JWT alapú hozzáférés rövid élettartamú bearer tokennel működik. A kliens a lejárt token helyett a POST /users/refresh végponton kérhet új hozzáférési tokent a korábban kapott refresh token felhasználásával.

4.4. Tesztesetek

A tesztek a Quiz.Test projektben találhatók. A megoldás külön egységteszteket és integrációs teszteket tartalmaz.

Tesztfájl	Típus	Lefedett működés
UserServiceTests	egységteszt	felhasználó létrehozása, bejelentkezés, aktuális felhasználó kezelése, kijelentkezés, jogosultságellenőrzés
QuizServiceTests	egységteszt	kvíz létrehozás, szerkesztés, publikálás, csatlakozás, kérdésléptetés, lezárás, üzenetküldés, befejezés
UsersControllerIntegrationTests	integrációs teszt	REST végpontok viselkedése felhasználókezelésnél, sikeres és hibás hitelesítési esetek
QuizControllerIntegrationTests	integrációs teszt	REST végpontok viselkedése kvízkezelésnél, jogosultsági és játékfolyamati esetek

A tesztek lefedik a sikeres és hibás bejelentkezést, a jogosultságellenőrzést, a kvízek létrehozását és szerkesztését, a publikálási és csatlakozási folyamatot, a kérdésléptetést, a kérdéslezárást, a kvíz befejezését és a chat üzenetküldést.

A legfontosabb funkcionális tesztesetek rövid leírása az alábbi táblázatban látható.

Teszt neve	Előfeltétel	Bemenet / művelet	Elvárt eredmény
Sikeres bejelentkezés	A felhasználó már regisztrált.	POST /users/login helyes email és jelszó megadásával.	A rendszer 200 OK választ ad vissza JWT tokennel, refresh tokennel és user azonosítóval.
Sikertelen bejelentkezés	A felhasználó már regisztrált.	POST /users/login hibás jelszóval.	A rendszer hibaválaszt ad, és nem ad ki tokent.
Token megújítása	A kliens rendelkezik érvényes refresh tokennel.	POST /users/refresh a refresh token elküldésével.	A rendszer új hozzáférési tokent ad vissza.
Kvíz létrehozása	A felhasználó hitelesítve van.	POST /quizzes érvényes címmel és kérdéslistával.	A rendszer létrehozza a kvízt és 201 Created választ ad vissza.
Kvíz szerkesztése publikálás előtt	A felhasználó a kvíz tulajdonosa, a kvíz még nincs publikálva.	POST /quizzes/{id}/update módosított címmel vagy kérdésekkel.	A rendszer elmenti a módosításokat és 202 Accepted választ ad vissza.
Csatlakozás közzétett kvízhez	Létezik publikált kvíz érvényes join code-dal.	POST /quizzes/join a join code és játékosnév megadásával.	A játékos bekerül a kvíz résztvevői közé.
Következő kérdés indítása	A kvíz publikált, a műveletet a tulajdonos végzi.	GET /quizzes/{id}/next meghívása.	Az aktuális kérdés indexe növekszik, és SignalR értesítés indul a kliensek felé.
Aktuális kérdés lezárása	A kvízben már elindult egy kérdés.	GET /quizzes/{id}/question/end meghívása.	Az aktuális kérdés lezárul, a helyes válasz indexe továbbításra kerül a klienseknek.
Chat üzenet küldése	A játékos már csatlakozott a kvízhez.	POST /quizzes/message a join code, játékosnév és üzenet megadásával.	Az üzenet elmentődik és valós időben megjelenik a többi kliens számára is.